

Athlete Intelligence — Complete Architecture Audit Report

Date: July 2026

Scope: Full read-only audit of AI pipeline, API routes, database schema, and frontend

Status: No code has been modified. Awaiting approval before any changes.

Table of Contents

Current Architecture — How Data Flows
Finding Index
Critical Findings
High Priority Findings
Medium Priority Findings
Low Priority Findings
Duplicate Services & Redundant Code
Where AI Makes Assumptions Instead of Using Evidence
Recommended Fix Sequence

1. Current Architecture

How an Athlete Gets Populated (the full data flow)

	gpt-5.6-luna used everywhere				
C2	No athlete identity verification — wrong person can be researched	AI Pipeline	Critical	High	
C3	Perplexity failure silently falls back to LLM hallucination	AI Pipeline	Critical	Medium	
C4	Hardcoded date in research prompt becomes incorrect immediately	AI Pipeline	Critical	Low	
C5	Discovery stage defaults sport to "Athletics" on any failure	AI Pipeline	Critical	Low	
H1	Citation index mismatch — source URLs attached to wrong intelligence items	AI Pipeline	High	High	
H2	Social follower counts are unverified estimates, not live data	Social	High	Medium	
H3	Photo backfill overwrites manually corrected photos	Photo	High	Low	
H4	No athlete name disambiguation — common names return wrong person	AI Pipeline	High	High	
H5	intelligence_count is a denormalised counter that drifts	Database	High	Low	
H6	No indexes on any foreign key columns in child tables	Database	High	Low	
H7	Timeline dates fall back to today's date on parse failure	AI Pipeline	High	Low	
H8	Duplicate repopulate endpoints (admin vs athlete route)	Architecture	High	Low	
M1	Three different model tiers used for identical social extraction task	AI Pipeline	Medium	Low	
M2	reasoning_effort: "none" applied to invalid model in photo lookup	AI Pipeline	Medium	Low	
M3	autoPopulateAthlete runs with no job tracking — lost on server restart	Architecture	Medium	High	
M4	Chat conversations not persisted despite database tables existing	Architecture	Medium	Medium	
M5	Dashboard makes redundant API calls already covered by /dashboard endpoint	Frontend	Medium	Low	
M6	Polling termination is time-based — 60s timeout causes "no data" for slow population	Frontend	Medium	Medium	
M7	No unique constraint on athlete name — duplicate athletes possible	Database	Medium	Low	
M8	Stripe subscription status relies entirely on webhook sync — missed webhooks = wrong status	Payments	Medium	Medium	
M9	All users share the same athlete roster — no multi-tenancy	Architecture	Medium	High	
L1	SSE streaming parse errors silently discarded in DossierPage	Frontend	Low	Low	
L2	as any type casting in DossierPage hides API contract violations	Frontend	Low	Low	
L3	Alert preferences stored but never actioned	Architecture	Low	High	
L4	conversations and messages tables unused dead weight	Database	Low	Low	
L5	Founder email hardcoded in 5 separate places	Architecture	Low	Low	

3. Critical Findings

C1 — Hardcoded model alias `gpt-5.6-luna` used in production paths

Category: AI Pipeline

Priority: Critical

Difficulty to fix: Low (change 3 strings)

Where it appears:

-

ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/l
ib/
a
ut
o-
p
o
p
ul
at
e.t
s
—
st
ru
ct
ur
e
d
d
at
a
ex
tr
ac
tio
n
(t
h
e
co
re
pi
p
eli
n
e)

-

ar
tif
ac
ts/
a
pi-
se
rv

er
/sr
c/l
ib/
p
h
ot
o-l
o
ok
u
p.t
s
—
P
er
pl
ex
ity
U
R
L
ex
tr
ac
tio
n
•
ar
tif
ac
ts/
a
pi-
se
rv
er
/sr
c/r
o
ut
es
/s
u
m
m
ar
y.t
s
—
d
os
si
er
s
u
m
m
ar
y
g
e
n
er
ati
o
n

Root cause:
gpt-5.6-luna is an undocumented internal model alias in the Replit AI proxy. It does not correspond to any

publicly documented OpenAI model. It was likely chosen during early development as a test alias and never changed. Because it is internal, it could be renamed, deprecated, or redirected to a different model at any time without notice. If it breaks, the entire auto-populate pipeline fails silently (the error is caught and swallowed), athletes appear with 0 intelligence items, and no error is surfaced to the user or admin.

Additionally: photo-lookup.ts sets reasoning_effort: "none" on this model. This parameter is only valid for OpenAI o-series reasoning models. On gpt-5.6-luna (or any GPT-4-class model), it is an unrecognised parameter. The API may silently ignore it, or it may cause unpredictable behaviour.

Evidence:

- a
ut
o-
p
o
p
ul
at
e.t
s:
m
o
d
el:
"
g
pt
-5
.6
-lu
n
a"
w
ith
m
ax
_c
o
m
pl
eti
o
n
_t
ok
e
ns
: 8
1
9
2
- p
h
ot
o-l
o
ok
u
p.t
s:
m
o
d
el:
"
g
pt
-5

.6
-lu
n
a"
, r
e
as
o
ni
n
g
—
ef
fo
rt:
"
n
o
n
e"
•
su
m
m
ar
y.t
s:
m
o
d
el:
"
g
pt
-5
.6
-lu
n
a"
fo
r s
tr
e
a
mi
n
g
d
os
si
er
g
e
n
er
ati
o
n
•
a
d
mi
n.
ts
so
ci
al
b
ac

kfi
ll
us
es
g
pt
-4
o-
mi
ni
—
in
co
ns
ist
e
nt
w
ith
t
h
e
a
b
ov
e

Recommended fix:

Replace all three instances with gpt-4o. For auto-populate.ts (extraction), gpt-4o is the correct choice — it has a large context window and strong JSON generation. For summary.ts, gpt-4o streaming works well. For photo-lookup.ts, gpt-4o-mini is sufficient (it only extracts a URL from a short text). Remove reasoning_effort: "none" entirely.

C2 — No athlete identity verification at any stage

Category: AI Pipeline

Priority: Critical

Difficulty to fix: High

Root cause:

The system never confirms it is researching the correct person. When a user types "Brook Macdonald", the following happens:

- Discovery stage sends only the name to GPT, which returns a sport/nationality guess from training data.

- There is no check that the GPT-identified sport matches a real athlete with that name.

- Perplexity research receives name + sport + nationality, but is not instructed to confirm identity before proceeding. If two athletes share a name (e.g., two "James Wilson"s), Perplexity may research the wrong one without warning.

- Extraction stage receives the Perplexity text and structures it — it never asks "is this the correct athlete?"

- No cross-validation exists at any point — no checking that discovered social handles actually belong to the named athlete, or that competition results match the athlete's known sport/nationality.

Evidence:

The Brook Macdonald Wikipedia photo bug documented in the project history (Perplexity returned results for a different person; the Wikipedia search returned Elijah Just) is a direct symptom of this missing identity layer.

Recommended fix:

Add an identity confirmation step between Perplexity research and GPT extraction. After Perplexity returns its research, send the first paragraph back to GPT with the question: "Does this text describe [name], [nationality] [sport] athlete? Answer YES or NO and explain briefly." Only proceed to full extraction if YES. This costs one extra API call but eliminates wrong-person data entirely.

For social handles: after extraction, verify each claimed handle by checking if the account name or bio contains the athlete's name before writing it to the DB.

C3 — Perplexity failure silently falls back to GPT training knowledge

Category: AI Pipeline

Priority: Critical

Difficulty to fix: Medium

Root cause:

In `auto-populate.ts`, if the Perplexity research call fails for any reason (timeout, API error, empty response), the code logs a warning and proceeds with an empty `researchText = ""`. The extraction prompt is then sent to `gpt-5.6-luna` with no research text at all. GPT does not know the research failed — it receives an empty input and fills in the JSON from its training data.

This means:

-

R
a
nk
in
gs
, r
es
ult
s,
a
n
d
co
m
p
eti
tio
n
d
at
a
ar
e
fa
br
ic
at
e
d
fr
o
m
tr
ai
ni
n
g
kn
o
wl
e
d
g
e
(w
hi
ch
h
as

a
kn
o
wl
e
d
g
e
cu
to
ff)

-

C

o
nfi
d
e
nc
e
sc
or
es
o
f
6
5
—
8
0
ar
e
as
si
g
n
e
d
to
fa
br
ic
at
e
di
te
m
s
—
t
h
e
sa
m
e
ra
n
g
e
as
lo
w-
co
nfi
d
e
nc
e
ve
rifi
e
di
te
m
s
•
T
h
e
us
er
h
as

n
o
w
ay
t
o
di
sti
n
g
ui
sh
r
e
al
d
at
a
fr
o
m
h
all
uc
in
at
e
d
d
at
a
•
T
h
e
at
hl
et
e
a
p
p
e
ar
s f
ull
y
p
o
p
ul
at
e
d
in
th
e
UI

Code path:

```
#!/ GPT invents data from training by research
#!/ All fabricated items written to DB
#!/ lastCrawledAt set (looks like a successful run)
```

Recommended fix:

If Perplexity fails, do not proceed to extraction. Set lastCrawledAt to null, set a new populationStatus field (e.g., "research_failed") on the athlete row, and surface this clearly in the admin Crawl tab. Retry Perplexity once after a 5-second delay before giving up.

C4 — Research prompt contains a hardcoded date that is immediately wrong

Category: AI Pipeline

Priority: Critical

Difficulty to fix: Low (one-line change)

Root cause:

"Today's date is July 29, 2026."

The Perplexity system prompt in auto-populate.ts contains:

This date was correct on the day the prompt was written. It is wrong for every subsequent day. Perplexity is a live web search engine — it uses the "today's date" context to determine what counts as "recent" results, to interpret relative time references ("last month", "current season"), and to anchor competition schedules. A wrong date causes:

- "

R
ec
e
nt
r
es
ult
s"

t
o i
nc
lu
d
e
ev
e
nt
s t
h
at
a
re
a
ct
u
all
y
m
o
nt
hs
ol
d

- "

U
pc
o
mi
n
g
co
m
p
eti
tio
ns
" t
o i
nc
lu
d
e
ev

ent
st
h
at
h
av
e
alr
e
a
dy
h
a
p
p
e
n
e
d
•
Ti
m
eli
n
e
a
nc
h
or
in
g
er
ro
rs
th
ro
u
g
h
o
ut

Recommended fix: ``${new Date().toISOString().split('T')[0]}``

Replace the hardcoded string with `new Date().toISOString().split('T')[0]` injected at call time:

C5 — Athlete discovery silently defaults to sport="Athletics" on failure

Category: AI Pipeline

Priority: Critical

Difficulty to fix: Low

Root cause: `typeof data.sport === "string" ? data.sport : "Athletics",`

`typeof data.event === "string" ? data.event : "",`

`typeof data.nationality === "string" ? data.nationality : ""`

In auto-populate.js, the `discoverAthleteProfile` function calls GPT to identify an athlete's sport from just their name. The fallback chain is:

If GPT returns anything other than a plain string (null, undefined, a number, a malformed JSON), the athlete is silently labelled as an "Athletics" athlete with no event and no nationality. This bad data is then passed to Perplexity as context, biasing the research toward athletics athletes with the same name.

Consequences:

•
A
cy
cli
n
g
at
hl
et
e
co
ul

d
b
e
re
se
ar
ch
e
d
as

a
tr
ac
k-
a
n
d-
fie
ld
at
hl
et
e

•
P
er
pl
ex
ity
, t
ol
d
to
r
es
e
ar
ch
"[
N
a
m
e]

,
At
hl
eti
cs
a
thl
et
e,
N
e
w
Z
e
al
a
n
d"
,
wi
ll f
in
d
th
e

wr
o
n
g
p
er
so
n
or
m
ix
e
d
d
at
a
•
T
h
e
us
er
s
e
es
s
p
or
t=
"A
thl
eti
cs
"
o
n
th
e
at
hl
et
e
ca
rd
w
ith
n
o i
n
di
ca
tio
n
a
ny
thi
n
g
w
e
nt
w
ro
n
g

Recommended fix:

If discovery returns unusable data (empty sport, empty nationality), reject the creation and return a 422 with:
"Could not identify [name] as a known athlete. Please provide sport and nationality." Do not create the stub row
with bad data.

4. High Priority Findings

H1 — Citation index mismatch corrupts source attribution

Category: AI Pipeline

Priority: High

Difficulty to fix: High

Root cause:

Perplexity returns research text with inline citation markers like [1], [2] plus a citations array of URLs. The extraction prompt tells GPT to use the citation index numbers as sourceUrl values. However:

- GPT does not have access to the actual citation URLs — it only sees the text with [1], [2] markers
- GPT is expected to map [3] !' citations[2] (0-indexed), but it frequently maps by topic match rather than index
- The index mapping is done entirely inside GPT's reasoning — it can and does produce wrong mappings
- A wrong mapping means an intelligence item about a sponsorship deal is attributed to a Wikipedia article about a different athlete

This is why source URLs in intelligence items are often incorrect or irrelevant.

[2] <https://nzoc.org.nz/>...

Recommended fix:

Then instruct GPT: "For sourceUrl, copy the exact URL from the numbered citations list above." Additionally, after extraction, validate that each sourceUrl actually appears in the citations list — reject any that don't.

H2 — Social media follower counts are unverified estimates

Category: Social Media

Priority: High

Difficulty to fix: Medium

Root cause:

Three separate code paths produce follower counts, and none of them are verifiably accurate:

Auto-populate uses perplexity/sonar (not sonar-pro), asks for social data as part of a broad research request, then passes the text to gpt-5.6-luna for extraction. Perplexity sonar may cite follower counts from articles that are months or years old.

Admin backfill / Dossier refresh uses perplexity/sonar-pro (higher quality), but still relies on Perplexity's cached web data, which can be days or weeks old. There is no validation that the handle discovered actually belongs to the athlete.

X API (Twitter only) is the only source of live, authoritative data — but it only runs if TWITTER_BEARER_TOKEN is set, which it currently is not. So all Twitter follower counts are Perplexity estimates.

For Instagram and TikTok, there are no official public APIs, so the data is always estimated. The problem is that the platform presents these numbers as facts without any indication they are estimates.

Recommended fix:

-

S
et
T
W
IT
T
E
R

B
E
A
R
E
R

T
O
K
E
N

to
g
et
r
e
al
T
wi
tt
er
d
at
a
•
L
a
b
el
es
ti
m
at
e
d
fol
lo
w
er
c
o
u
nt
s
in
th
e
UI
(
e.
g.
,
""H
4
5
K
(e
sti
m
at
e
d)
")
a
n
d
sh
o
w
th
e
d
at
a
re
tri
ev
al
d
at

e
•
A
d
d
a
so
ci
al
_
d
at
a
_r
et
ri
ev
e
d
_
at
ti
m
es
ta
m
p
co
lu
m
n
to
a
thl
et
es
•
C
o
ns
id
er
a
d
di
n
g
a
so
ci
al
_c
o
nfi
d
e
nc
e
fie
ld
(0
_
1
0
0)
t
o i
n
di

cate
wh
et
her
the
count

is
from
man
an
API
calls
or
an
LLM
estima
te

H3 — Photo backfill overwrites manually corrected photos

Category: Photo
Priority: High
Difficulty to fix: Low

Root cause:

The admin "Backfill Photos" button calls POST /api/admin/backfill-photos. This route queries all athletes where avatarUrl IS NULL. However, if an admin manually patches a photo (e.g., via direct DB SQL), then later clicks "Backfill Photos" again for athletes without photos, athletes who had their photo overwritten from a previous bad backfill will not be re-patched. But if a photo was patched to a non-null value, it will be skipped.

The real problem documented in the project history: running backfill overwrote Brook Macdonald's manually-corrected photo with a wrong one. This means the IS NULL check alone is insufficient — any photo found by the algorithm, even a wrong one, is written without confirmation.

Additionally, there is no avatar_url_locked flag, so there is no way to protect a manually-verified photo from future overwrites.

Recommended fix:

Add a boolean column avatar_url_locked to athletes (default false). The backfill route should skip athletes where avatar_url_locked = true. When an admin manually sets a photo via the UI, set avatar_url_locked = true automatically.

H4 — Common athlete names return ambiguous or wrong research

Category: AI Pipeline
Priority: High
Difficulty to fix: High

Root cause:

Name disambiguation is left entirely to Perplexity with no structural support. The research prompt passes name, sport, event, and nationality — but Perplexity's search results for common names (e.g., "James Wilson", "Chris Martin") can blend results from multiple people with similar names. Unlike a structured database lookup,

Perplexity returns the most prominent result for the name, which may not be the correct athlete.

The problem compounds with Wikipedia photo lookup: the Wikipedia search API returns results by relevance, not by exact match. An athlete with a common name whose Wikipedia page is less prominent than a same-named celebrity will receive the wrong photo.

Recommended fix:

Enrich the disambiguation context in the research prompt with every available identifier: known team, known coach, known career milestone. After discovery, ask the user to confirm the identified athlete description before running the full pipeline (a quick "Is this the right person?" confirmation step in the NewAgentPage UI).

H5 — `intelligence\_count` denormalised counter drifts from reality

Category: Database

Priority: High

Difficulty to fix: Low

Root cause:

athletes.intelligence_count is manually maintained:

-

S
et
t
o
th
e
co
u
nt
o
f i
ns
er
te
d i
te
m
s
at
e
n
d
of
a
ut
o
P
o
p
ul
at
e
At
hl
et
e

-

R
es
et
t
o
O
in
re
p
o
p
ul
at

er
utes
•
Never
update
data
ended
when
insert
divi
dual
items
sare
added
ded
out
side
ethe
pipe
elin
e

There is no database trigger or view keeping this in sync. If any code path inserts or deletes intelligence items without updating this counter, the displayed count will be wrong. This column is used in the admin Crawl tab to identify athletes needing repopulation (red badge if 0), making incorrect counts directly misleading.

Recommended fix:

Replace intelligence_count with a computed field: `SELECT COUNT(*) FROM intelligence_items WHERE athlete_id = $1`. This can be added as a subquery in the GET /athletes/:id route with negligible performance cost given proper indexing (see H6). Alternatively, add a Postgres trigger to keep the count in sync.

H6 — No indexes on foreign key columns in child tables

Category: Database

Priority: High

Difficulty to fix: Low

Root cause:

Every child table (intelligence_items, timeline_events, contacts, competitions) has an athlete_id foreign key column. None of them have an index on this column. As the roster grows, every GET /athletes/:id/intelligence query performs a full sequential scan of the intelligence_items table. With 10 athletes × 15 intelligence items = 150 rows this is invisible. At scale (100 athletes × 15 items = 1,500 rows, or with historical data accumulating over months), query time degrades linearly.

Missing indexes:

• i

nt
ell
ig
e
nc
e
_it
e
m
s.
at
hl
et
e
_i
d
• i
nt
ell
ig
e
nc
e
_it
e
m
s.
di
sc
ov
er
e
d
—
at
(
us
e
d
fo
r
gl
o
b
al
or
d
er
in
g i
n
/in
tel
lig
e
nc
e)
• t
im
eli
n
e
—
ev
e
nt
s.
at
hl

et
e
_i
d
•
co
nt
ac
ts.
at
hl
et
e
_i
d
•
co
m
p
eti
tio
ns
.a
thl
et
e
_i
d
•
co
m
p
eti
tio
ns
.d
at
e
(u
se
d
fo
r fi
lte
ri
n
g
u
pc
o
mi
n
g
vs
p
as
t)
•
at
hl
et
es
.n
a
m
e
(u
se
d

for duplicating schema)

Recommended fix:

Add indexes via a Drizzle migration. All of these are single-column btree indexes — straightforward to add, zero application code changes required.

H7 — Timeline event dates default to today on any parse failure

Category: AI Pipeline

Priority: High

Difficulty to fix: Low

Root cause:

`date = item.date ? new Date(item.date).toISOString().split("T")[0] : today`
In `auto-populate.ts`, timeline event dates extracted from GPT are processed as:

Where `today` is the current date at the time of the pipeline run. If GPT returns a date that `new Date()` cannot parse (e.g., "Summer 2019", "circa 2018", "unknown"), the date silently becomes `today`. The user then sees a career event from "today" that actually happened years ago, breaking the chronological timeline.

Evidence: Timeline events with incorrect dates cluster around the date of the last repopulate run, which is a diagnostic signal for this bug.

Recommended fix:

If a date cannot be parsed to a valid ISO date, skip that timeline event entirely or store it with a `date_approximate` boolean and a `date_display` text field for human-readable imprecise dates like "2019". Do not substitute today's date.

H8 — Two identical repopulate endpoints exist

Category: Architecture

Priority: High

Difficulty to fix: Low

Root cause:

`POST /api/admin/repopulate/:id` and `POST /api/athletes/:id/repopulate` perform exactly the same operation: delete all child records for the athlete, reset `lastCrawledAt` and `intelligenceCount` to `null/0`, and fire `autoPopulateAthlete` in the background. The only difference is the auth gate (admin vs standard user).

Having two endpoints for the same operation means:

-

Any bug fix must update

d
in
tw
o
pl
ac
es

•

L
o
gi
c
h
as
al
re
a
dy
di
ve
rg
e
d
sli
g
htl
y
b
et
w
e
e
n
th
e
tw
o

•

F
ut
ur
e
d
ev
el
o
p
er
s
wi
ll
b
e
co
nf
us
e
d
a
b
o
ut
w
hi
ch
t
o
ca
ll

Recommended fix:
Extract the repopulate logic into a shared service function `repopulateAthlete(id)`. Both routes call this function. The admin route additionally bypasses any per-user rate limiting that should be added to the standard route.

5. Medium Priority Findings

M1 — Three different model tiers used for the same social extraction task

Category: AI Pipeline
Priority: Medium
Difficulty to fix: Low
Evidence:

Code path	Research model	Extraction model
Auto-populate (new athlete)	perplexity/sonar	gpt-5.6-luna
Admin backfill social	perplexity/sonar-pro	gpt-4o-mini
Dossier "Refresh from Web"	perplexity/sonar-pro	gpt-4o-mini

The auto-populate path uses the cheaper, less capable perplexity/sonar (not sonar-pro) and the unknown gpt-5.6-luna model. The manual refresh paths use sonar-pro + gpt-4o-mini. This means an athlete's initial social data (set at creation) is extracted with lower quality tools than a manual refresh. Users who click "Refresh from Web" will get better data than the initial auto-populate — an inconsistency that will be confusing.

Recommended fix:
Standardise all social extraction on perplexity/sonar-pro + gpt-4o-mini. Use a single shared `extractSocialMediaData(athleteName)` function called from all three paths.

M2 — `reasoning\_effort: "none"` applied to a non-reasoning model

Category: AI Pipeline
Priority: Medium
Difficulty to fix: Low
Evidence:
Root cause:

`reasoning_effort: "none"` is a parameter used by OpenAI for reasoning models (o1, o3, etc.). It is not a valid parameter for GPT-4-class models. The API may silently ignore it, or it may cause unexpected behaviour or errors on the model proxy. At minimum, it is dead code that signals confusion about which model is being used.

Recommended fix:
Change model to gpt-4o-mini and remove `reasoning_effort`. This task (extract a URL from a short paragraph) does not require a powerful model.

M3 — `autoPopulateAthlete` runs detached with no recovery mechanism

Category: Architecture
Priority: Medium
Difficulty to fix: High
Evidence:
Root cause:

The auto-populate pipeline is fired as a detached background promise. There is no job queue, no persistence of job state, and no retry mechanism. If the server restarts, crashes, or is redeployed while a populate is in progress, the job is lost permanently. The athlete remains with 0 intelligence items and no lastCrawledAt, and there is no automatic retry. The admin must notice the athlete in the Crawl tab and manually click Repopulate.

Recommended fix:
Add a `population_status` column to athletes with values: pending, in_progress, complete, failed. Set it to pending when the job starts, in_progress at the first Perplexity call, complete on success, failed on catch. On server startup, query for any athletes with in_progress status and re-trigger them. This provides basic job durability without a full queue system.

M4 — Chat conversations not persisted despite DB tables existing

Category: Architecture

Priority: Medium

Difficulty to fix: Medium

Root cause:

The database has conversations and messages tables. The chat route (chat.ts) does not write to either table. All chat history is held in React component state in ChatPage.tsx and is lost on page navigation or refresh. The database tables are dead weight.

Consequence: Users cannot review past conversations, and the platform cannot learn from past queries or build a user history feature.

Recommended fix:

Either (a) wire up the chat route to persist messages to the DB (create/reuse a conversation, insert each message as it is sent/received), or (b) drop the unused tables to reduce schema confusion. Option (a) is the right long-term choice.

M5 — Dashboard makes redundant API calls

Category: Frontend

Priority: Medium

Difficulty to fix: Low

Root cause:

Dashboard.tsx makes three parallel API calls:

- useGetDashboard ! GET /api/dashboard (returns aggregate stats + recent intelligence + priority athletes)

- useListAthletes ! GET /api/athletes (returns full athlete list)

- useListIntelligence ! GET /api/intelligence (returns last 50 intelligence items across roster)

The GET /api/dashboard response already includes recentIntelligence and priorityAthletes. Calls 2 and 3 duplicate data already returned by call 1, resulting in 3 network requests where 1 would suffice.

Recommended fix:

Audit what data the Dashboard actually renders from each call. If calls 2 and 3 are providing data already in the dashboard response, remove the redundant hooks.

M6 — Polling termination is time-based, causing false "no data" states

Category: Frontend

Priority: Medium

Difficulty to fix: Medium

Root cause:

DossierPage.tsx has two polling loops:

- Initial populate polling: polls every 3s for a maximum of 60 seconds (20 attempts). If auto-populate takes longer than 60s (which it routinely does for athletes with sparse public data — Perplexity alone can take 15–30s, and GPT extraction adds another 10–20s), the polling stops and the dossier shows empty tabs. The user has no feedback that the populate is still running in the background.

- Refresh polling: polls every 3s for up to 180s (60 attempts). Same issue at scale.

Neither loop differentiates between "still running" and "genuinely no data found".

Recommended fix:

Add a population_status field to the athlete API response (from finding M3). The polling loop should terminate on status === "complete" or status === "failed", not on a timeout. Show a spinner with status text ("Researching athlete...", "Extracting data...") while status === "in_progress".

M7 — No unique constraint on athlete name

Category: Database

Priority: Medium

Difficulty to fix: Low

Root cause:

The athletes table has no unique constraint on name. The POST /athletes/discover route does perform a duplicate check in application code, but:

- There is a race condition: two simultaneous requests for the same name will both pass the check before either inserts

- The check is case-sensitive — "Brook Macdonald" and "brook macdonald" would create duplicates

- If the check is bypassed (e.g., via POST /athletes directly), a duplicate is created

Recommended fix:

Add a unique index on LOWER(name) in the Drizzle schema. The duplicate check in application code can remain as a friendly user-facing message.

M8 — Stripe subscription status relies entirely on webhook delivery

Category: Payments

Priority: Medium

Difficulty to fix: Medium

Root cause:

GET /api/stripe/subscription reads from stripe.subscriptions (the local sync table), not from the Stripe API. If a Stripe webhook is missed, delayed, or fails delivery:

-

A
us
er
w
h
o
ca
nc
el
s t
h
eir
s
u
bs
cri
pti
o
n
co
nti
nu
es
t
o
a
p
p
e
ar
a
s
ac
tiv
e

-

A
us
er
w
h
o
u
p
gr
a
d
es
c
o
nti
nu
es
t
o

a
p
p
e
ar
o
n
th
e
ol
d
pl
a
n
•
A
n
e
w
su
bs
cri
b
er
a
p
p
e
ar
s
as
h
av
in
g
n
o
su
bs
cri
pti
o
n
u
nti
l t
h
e
w
e
b
h
o
ok
a
rri
ve
s

Stripe's own dashboard shows delivery failures are not uncommon under high load or during server downtime.

Recommended fix:

For the subscription status endpoint specifically, add a fallback that calls the Stripe API directly if the local record is more than 24 hours old or if the local status is ambiguous. Cache the result for a short period (5 minutes) to avoid excessive Stripe API calls.

M9 — No multi-tenancy — all users share one roster

Category: Architecture

Priority: Medium
Difficulty to fix: High

Root cause:

Every authenticated user sees the same athletes table — there is no `user_id` or `org_id` column filtering rows by owner. This is intentional for the current single-agency deployment model, but means:

- If a session is ended, a new session is created, and the user is logged in, they will see the first agency's athletes.

- Any user can delete any athlete.

- This

er
e
is
n
o
co
nc
e
pt
o
f"
m
y
at
hl
et
es
"
vs
"
sh
ar
e
d
at
hl
et
es
"

Recommended fix:

Add an `org_id` column to athletes (nullable initially for backward compatibility). When a user creates an athlete, set `org_id` to their Clerk organisation ID (or user ID for solo accounts). Apply a `WHERE org_id = $userOrgId` filter in all athlete list/fetch routes. This is a significant migration but is the correct foundation for multi-tenant SaaS.

6. Low Priority Findings

L1 — SSE streaming parse errors silently discarded

Category: Frontend

Priority: Low

Difficulty to fix: Low

```
try { ... } JSON.parse(line) ... } catch {}
```

In `DossierPage.tsx`, the SSE summary stream is parsed line-by-line with `try/catch` that swallows all errors: A malformed SSE line (e.g., from a partial flush or encoding issue) is silently dropped. If the `{ done: true }` event is corrupted, the loading state never clears.

Fix: Log the parse error and, if the stream ends without a done event being received, force the loading state to false after a timeout.

L2 — `as any` type casting hides API contract violations

Category: Frontend

Priority: Low

Difficulty to fix: Low

```
(athleteData as any)?.athlete ?? (athleteData as any)
```

```
Difficulty to fix: Low
```

```
(athleteData as any)?.items ?? []
```

`DossierPage.tsx` uses pattern type checking. If the API changes its response shape, TypeScript will not catch the breakage — the page will silently render empty or wrong data.

Fix: Use the generated types from `@workspace/api-client-react` directly. If the generated types are wrong, fix the OpenAPI spec rather than casting.

L3 — Alert preferences stored but never actioned

Category: Architecture

Priority: Low

Difficulty to fix: High

alert_configs stores per-athlete notification preferences (enabled, frequency per category). No delivery mechanism exists. The UI implies alerts will be sent to users, which is currently false.

Fix: Implement a scheduled job (Replit scheduled deployment or cron) that queries for new intelligence items, checks alert_configs, and sends emails via a transactional email service (Resend is the simplest integration).

L4 — `conversations` and `messages` tables are unused dead weight

Category: Database

Priority: Low

Difficulty to fix: Low

These tables exist in the schema but no route reads from or writes to them. They consume schema space and confuse any engineer reading the schema.

Fix: Either wire them up (see M4) or drop them with a migration.

L5 — Founder email hardcoded in 5 places

Category: Architecture

Priority: Low

Difficulty to fix: Low

anigemmill@theoutsidein.nz appears in:

- App.tsx — sidebar visibility
- AdminPage.tsx — admin page guard
- Sidebar.tsx — admin nav item
- admin.ts — requireAdmin middleware
- stripe.ts — FOUNDER_EMAIL constant

If the email ever changes, all 5 must be updated in sync. A missed update would lock the admin out of their own panel.

Fix: Move to a FOUNDER_EMAIL Replit secret. Read it in API code via process.env.FOUNDER_EMAIL. Pass it to the frontend via a GET /api/config endpoint (public, non-sensitive) so the frontend can compare without hardcoding.

7. Duplicate Services & Redundant Code

Duplication	Location A	Location B	Impact
Athlete repopulate logic	admin.ts: POST /repopulate/:id	athletes.ts: POST /:id/repopulate	Bug fixes must be applied twice; logic has already diverged
Social media extraction	admin.ts: backfill-social	athletes.ts: refresh-social	Same Perplexity+GPT pattern copy-pasted; model inconsistency
Dashboard data	GET /api/dashboard	useListAthletes + useListIntelligence in Dashboard.tsx	3 API calls where 1 suffices
Model selection	gpt-5.6-luna in 3 files	gpt-4o-mini in 2 files	No single source of truth for model configuration
Auth token injection	custom-fetch.ts (React Query)	useAuthFetch.ts (manual fetch)	Two patterns for the same thing; future engineers will use both inconsistently

8. AI Assumptions Inventory

This section catalogues every place the AI makes an assumption rather than using verified evidence.

#	Assumption	Where	Risk
---	------------	-------	------

"This athlete's sport is X" — based
on GPT training knowledge of the
name

Discovery stage

			Wrong sport !' wrong research !' wrong data throughout
A2	"The Perplexity result describes the correct athlete"	All research stages	No verification; wrong person's data stored
A3	"Citation [3] in the text refers to citations[2] in the array"	Extraction stage	Source URLs attached to wrong intelligence items
A4	"This follower count is current"	Social extraction	Perplexity may cite articles with counts from years ago
A5	"This Instagram handle belongs to this athlete"	Social extraction	No cross-check; could assign another person's handle
A6	"This competition result is for the athlete's primary event"	Extraction stage	Mix of events for multi-discipline athletes
A7	"Today is July 29, 2026"	Research prompt	Stale date causes temporal errors in all research
A8	"A confidence score of 80 means this item is reliable"	All extracted data	Confidence scores are GPT self-assessments, not externally validated
A9	"If Perplexity returns nothing, GPT training data is an acceptable substitute"	Pipeline fallback	Fabricated data stored as if verified
A10	"The timeline event date 'Summer 2019' should become today's date"	Timeline parsing	Breaks chronological timeline; events appear as recent
A11	"The first Wikipedia search result for this name is the correct athlete"	Photo lookup	Different athlete's photo assigned
A12	"A photo URL returned by Perplexity is a direct image link"	Photo lookup	Perplexity may return a page URL, not an image URL

9. Recommended Fix Sequence

The following order minimises rework and addresses the most impactful problems first.

Phase 1 — Data Integrity (fix before any new features)

Step	Finding	Why first
1	Fix hardcoded date in prompt (C4)	One-line change; immediately improves every new populate run
2	Replace gpt-5.6-luna with gpt-4o / gpt-4o-mini (C1)	Eliminates dependency on undocumented alias; removes broken reasoning_effort param
3	Abort pipeline if Perplexity fails (C3)	Stops fabricated data being written to DB
4	Handle unknown discovery gracefully — don't default to "Athletics" (C5)	Prevents bad identity context poisoning the research
5	Fix timeline date fallback — skip instead of defaulting to today (H7)	Immediate timeline accuracy improvement

Phase 2 — Database Health (no user-facing changes, big reliability gain)

Step	Finding	Why
6	Add indexes on all FK columns + name + discovered_at (H6)	Required before any scale; zero risk migration
7	Add unique index on LOWER(athletes.name) (M7)	Prevents duplicate athletes
8	Replace intelligence_count with computed query (H5)	Eliminates counter drift permanently
9	Add avatar_url_locked column (H3)	Protects manually verified photos

Phase 3 — AI Quality (meaningful accuracy improvements)

Step	Finding	Why
10	Add identity confirmation step after Perplexity research (C2, H4)	Eliminates wrong-person data at source
11	Fix citation mapping — pre-format citations into prompt (H1)	Correct source attribution
12	Standardise social extraction to sonar-pro + gpt-4o-mini (M1, M2)	Consistent, known-good quality
13	Add social_data_retrieved_at and label estimates in UI (H2)	Honest data presentation

Phase 4 — Architecture (operational reliability)

Step	Finding	Why
14	Add population_status field; recover in-progress jobs on startup (M3, M6)	Job durability
15	Merge duplicate repopulate endpoints into shared service (H8)	Single source of truth
16	Persist chat conversations to DB (M4)	Activates existing schema
17	Move founder email to env secret (L5)	Operational safety
18	Fix dashboard redundant API calls (M5)	Performance

End of audit. No code has been modified. All findings are based on static analysis of the codebase as of July 2026. Awaiting approval before any changes are made.